

07- Search Engines

What is a Search Engine?

A search engine is a tool or an online service that allows users to search for information on the Internet.

Search Engine use Keyword Searching Algorithms to search queries and give output.

The primary function of a search engine is to help users find relevant documents, web pages, images, videos, or other types of content based on their queries or keywords.

Search engines use sophisticated algorithms to index and rank the vast amount of information available on the web, making it easier for users to access the most relevant and useful results.

How Search Engines Work?

1. Web Crawling:

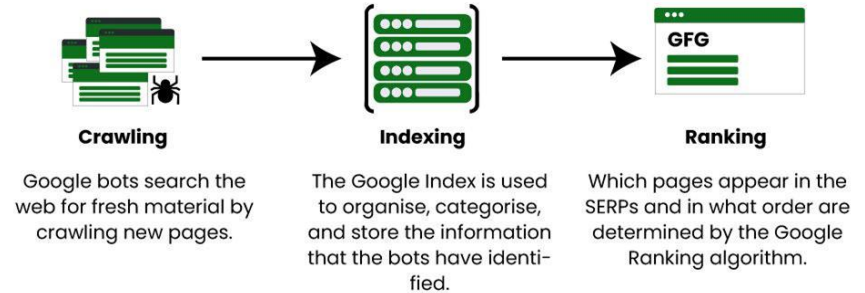
- Search engines use automated programs called [web crawlers](#) or spiders to browse the web and systematically scan websites for information.
- These crawlers follow links from one page to another, indexing the content of each page they visit.

2. [Indexing](#):

- The information collected by web crawlers is organized and stored in a database called an index.
- The index contains information about the content, keywords, and structure of web pages, making it easier and faster to retrieve relevant results.

3. Ranking Algorithms:

- Search engines employ complex keyword searching algorithms to analyze and rank the indexed pages based on their relevance to a user's query.
- Factors such as keyword relevance, page quality, user engagement, and other criteria are used to determine the order in which results are presented.



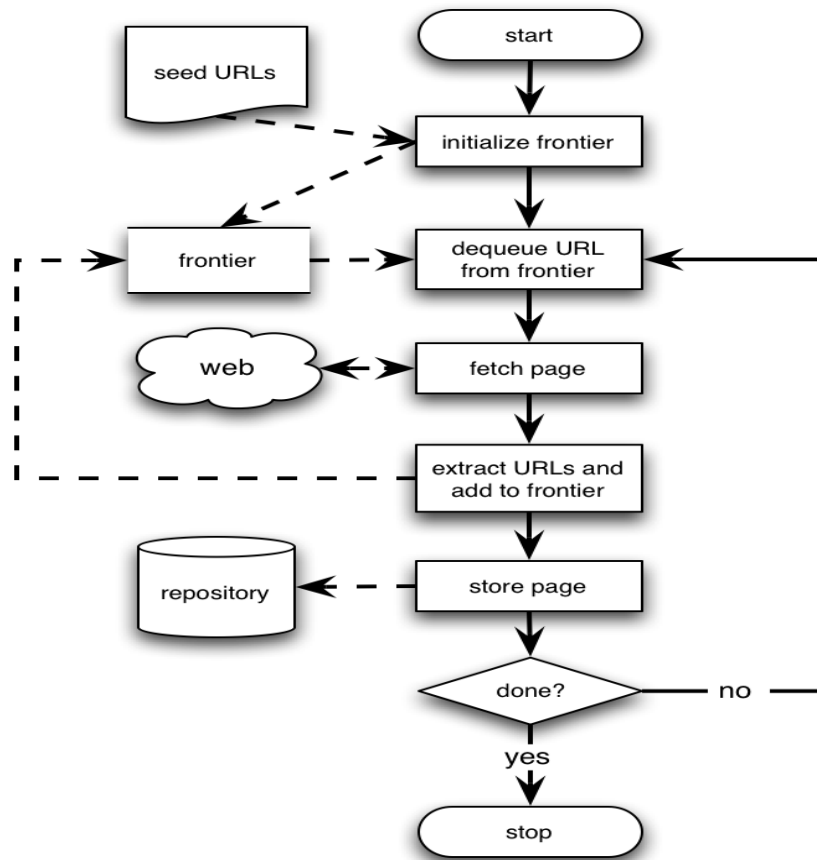
Many names

- Crawler
- Spider
- Robot (or bot)
- Web agent
- Wanderer, worm, ...
- And famous instances: googlebot, scooter, slurp, msnbot, ...

Overview of Search Engine Indexing Techniques

Search Engine Indexing techniques are diverse, each with unique mechanisms that contribute to efficient indexing. Below are some essential techniques that stand at the core of Search Engine Indexing:

- **Link Analysis:** By examining the network of links that connect different web pages, search engines can discern how pages relate to each other and rank their importance. This technique forms the foundation of algorithms like Google's PageRank.
- **Keyword Analysis:** This technique revolves around identifying the frequency and placement of keywords within web pages, providing search engines with clues about the page's content. It forms the basis for algorithms such as TF-IDF.
- **Content Analysis:** Some search engines deploy machine learning and natural language processing techniques to understand the context surrounding keywords, enabling them to deliver more relevant results to users.
- **Social Media Signals:** Sharing and liking the content on social media platforms is considered as an endorsement which could influence SERP position.
- **Parsed Data Thrusting:** Specific parsed data is given priority during the indexing, resulting in efficient searching.



Basic crawlers

- This is a **sequential** crawler
- **Seeds** can be any list of starting URLs
- Order of page visits is determined by **frontier** (**List/queue/stack**) data structure
- **Stop** criterion can be anything

Graph traversal (BFS or DFS?)

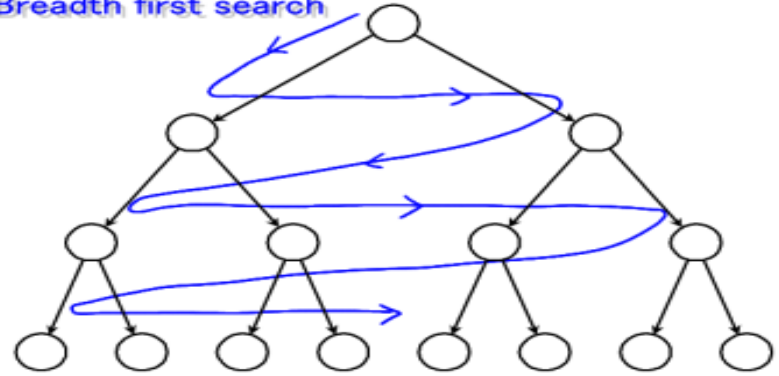
- **Breadth First Search**

- Implemented with QUEUE (FIFO)
- Finds pages along shortest paths
- If we start with “good” pages, this keeps us close; maybe other good stuff...

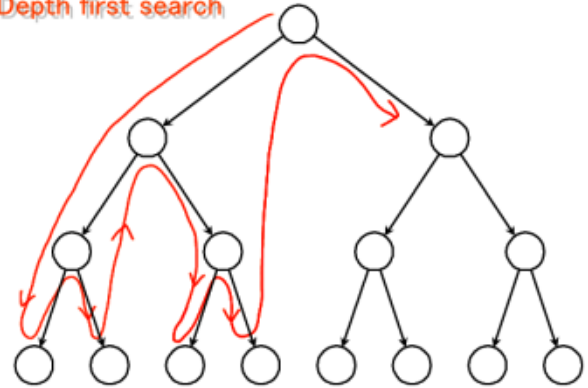
- **Depth First Search**

- Implemented with STACK (LIFO)
- Wander away (“lost in cyberspace”)

Breadth first search



Depth first search



Need Of **Keyword** Searching Algorithms

Search engines use keyword searching algorithms primarily because they provide an effective and efficient way to retrieve relevant information from vast amounts of data available on the web. The use of keywords as a basis for searching has several advantages:

1. Simplicity and User Familiarity:

- Keywords are simple and widely understood by users. People are accustomed to expressing their information needs in the form of words or phrases. Using keywords makes the search process intuitive and user-friendly.

2. Scalability:

- The web is enormous, with an immense volume of content. Keyword searching algorithms allow search engines to scale their operations efficiently. It's a practical approach for indexing and retrieving information from billions of web pages.

3. Information Retrieval Speed:

- Keyword searching algorithms enable quick retrieval of relevant results. By indexing and organizing web pages based on keywords, search engines can rapidly identify and present results matching the user's query.

4. Flexibility:

- Users can formulate queries in various ways using different combinations of keywords. Search engines are designed to handle a wide range of queries, making keyword searching algorithms a flexible and adaptable approach.

Need Of **Keyword** Searching Algorithms-2

5. **Relevance Ranking:**

- Keyword algorithms enable search engines to rank results based on relevance to the query. Sophisticated ranking algorithms consider factors such as the frequency of keywords, their placement, and the overall content quality to provide users with the most relevant results.

6. **Query Expansion:**

- Search engines often employ query expansion techniques to improve search results. For example, if a user's query lacks specificity, the search engine may expand it by adding related terms to refine the search.

7. **Adaptability to Natural Language:**

- While users often input queries as keywords, search engines have evolved to understand natural language to some extent. Advanced algorithms use natural language processing (NLP) keyword searching algorithms techniques to enhance the understanding of user queries.

8. **Historical Data and User Behavior:**

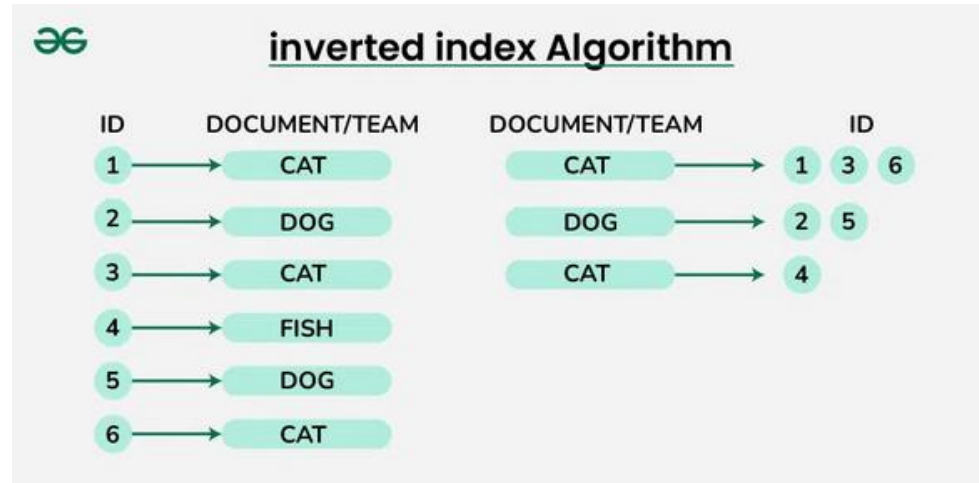
- Keyword searching algorithms enable search engines to analyze historical data and user behavior. This information is valuable for improving search results and personalizing recommendations for individual users

Keywords Searching Algorithms For Search Engine

Here are some keyword searching algorithms used by Search Engine for keyword searching:

1. Inverted Index :

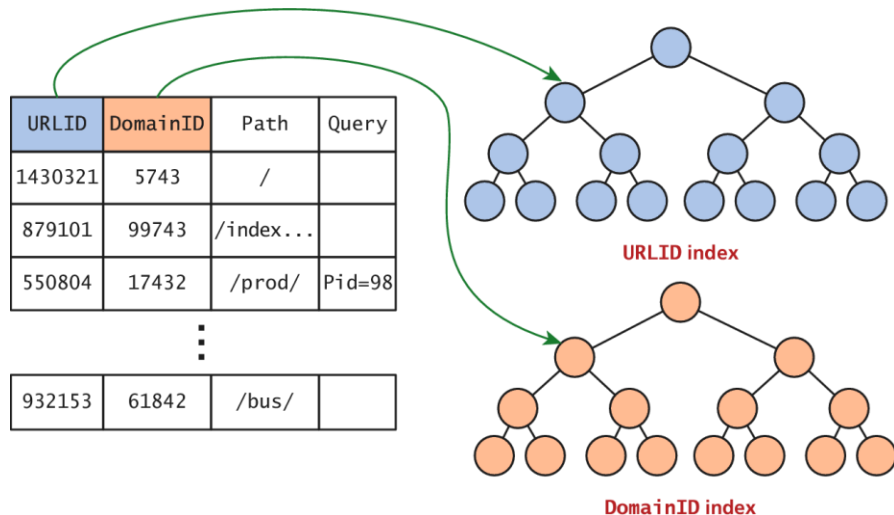
- **Algorithm Overview:** An inverted index Keywords Searching Algorithm is a data structure that maps keywords to the documents or web pages in which they appear. It involves creating an index of terms with pointers to the documents that contain them.
- **Significance:** Enables quick and efficient retrieval of documents containing specific keywords, forming the backbone of many search engine algorithms.



Indexing and Reverse Indexing

That which makes trillion URL searches possible

To understand indexing, consider what a crawler and a scraper might identify from a web page and how they might store it.



Indexing and Reverse Indexing

That which makes trillion URL searches possible

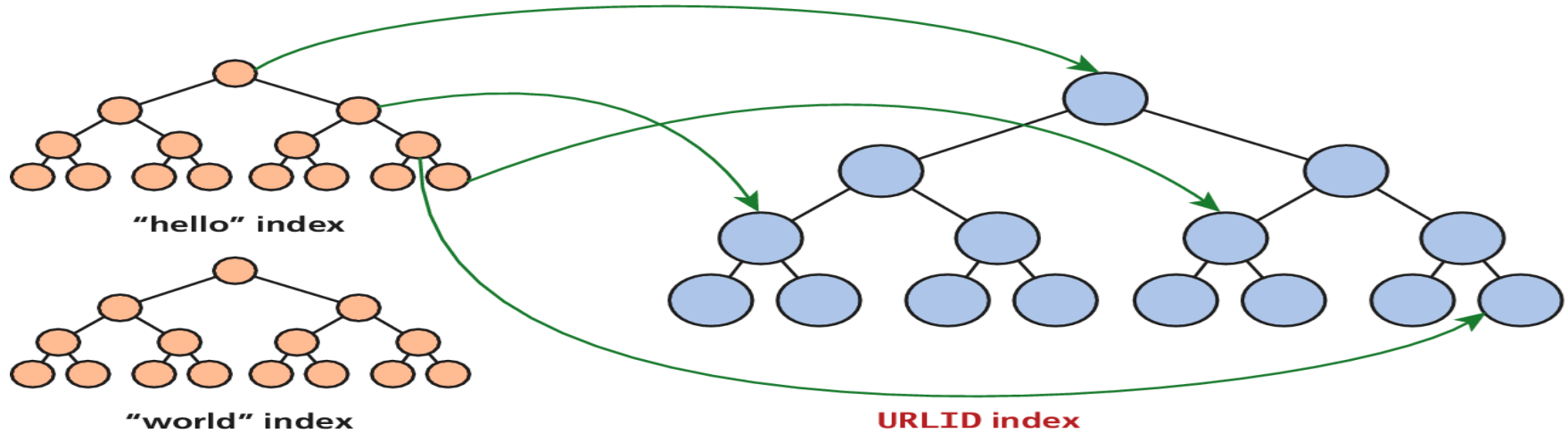
A **reverse index** (**Inverted index**) essentially indexes the **words**, rather than the **URLs**.

The mechanics of how this is done are **not standardized**, but generally word tables are created (for every word found in pages) so that each **word can be referenced by a unique integer (word ID)**, and indexes of these references can be built for faster searches.

Since there are tens of thousands of words, and each word might appear in millions of web pages, the demands on these indexes far exceed what a single database server can support.

Indexing and Reverse Indexing

That which makes trillion URL searches possible



Inverted Index types

<https://nullwords.wordpress.com/2013/04/18/inverted-indexes-inside-how-search-engines-work/>

There are two versions of an inverted index:

1. **Record-level index** which tells you which documents contain the term
2. **Fully inverted index** which tells you both the **document** a term is contained in and **exact position inside that file** it is (**line** number, index of **word** per line)

Inverted Index example-1

<https://nullwords.wordpress.com/2013/04/18/inverted-indexes-inside-how-search-engines-work/>

Assume 3 documents {0}, {1}, and {2} like the following:

{0} - "Turtles love pizza"

{1} - "I love my turtles"

{2} - "My pizza is good"

Then you would store them in a Inverted Indexes like this, note in fully inverted it lists the document index and the word index, for example the record (1,3) for "turtles" means it's in document 1 at the word of index 3 at that document:

	Record Level	Fully Inverted
"turtles"	{0, 1}	{ (0, 0), (1, 3) }
"love"	{0, 1}	{ (0, 1), (1, 1) }
"pizza"	{0, 2}	{ (0, 2), (2, 1) }
"i"	{1}	{ (1, 0) }
"my"	{1, 2}	{ (1, 2), (2, 0) }
"is"	{2}	{ (2, 2) }
"good"	{2}	{ (2, 3) }

Inverted Index example-2

<https://nullwords.wordpress.com/2013/04/18/inverted-indexes-inside-how-search-engines-work/>

So now if you wanted to search all three documents for the words

“my turtles”

Inside a **record level**, you would grab the sets (looking at record level only):

"turtles" {0, 1}

"my" {1, 2}

Then you would **intersect** those sets, coming up with the only matching set being **1**.

Using the **Fully Inverted Index** would also let us know that the word “my” appeared at position 2 and the word “turtles” at position 3. assuming the word position is important your search.

	Record Level	Fully Inverted
"turtles"	{0, 1}	{ (0, 0), (1, 3) }
"love"	{0, 1}	{ (0, 1), (1, 1) }
"pizza"	{0, 2}	{ (0, 2), (2, 1) }
"i"	{1}	{ (1, 0) }
"my"	{1, 2}	{ (1, 2), (2, 0) }
"is"	{2}	{ (2, 2) }
"good"	{2}	{ (2, 3) }

Note: للاطلاع فقط there is a sample c# code for inverted index at <https://nullwords.wordpress.com/2013/04/18/inverted-indexes-inside-how-search->

Keywords Searching Algorithms For Search Engine

2. TF-IDF (Term Frequency-Inverse Document Frequency):

- **Algorithm Overview:** In TF-IDF(Term Frequency - Inverse Document Frequency)Keyword Searching algorithms we assign weights to each term in a document based on its frequency (TF) in the document and its rarity (IDF) across the entire document collection.
- **Significance:** Helps prioritize terms that are important in a specific document but not common across all documents, enhancing the relevance of search results.

TF-IDF

$$W_{x,y} = tf_{x,y} \times \log \left(\frac{N}{df_x} \right)$$

TF-IDF

Term x within document y

$tf_{x,y}$ = frequency of x in y

df_x = number of documents containing x

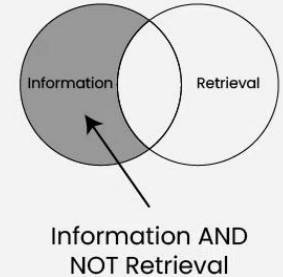
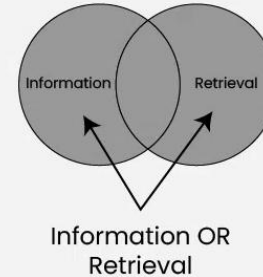
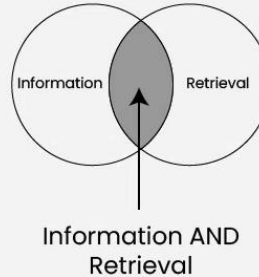
N = total number of documents

Keywords Searching Algorithms For Search Engine

3. Boolean Retrieval Model:

- **Algorithm Overview:** Uses Boolean operators (AND, OR, NOT) to combine keywords in search queries. Documents are retrieved based on whether they satisfy the Boolean conditions.
- **Significance:** Provides a foundation for Boolean search queries, allowing users to express complex search criteria.

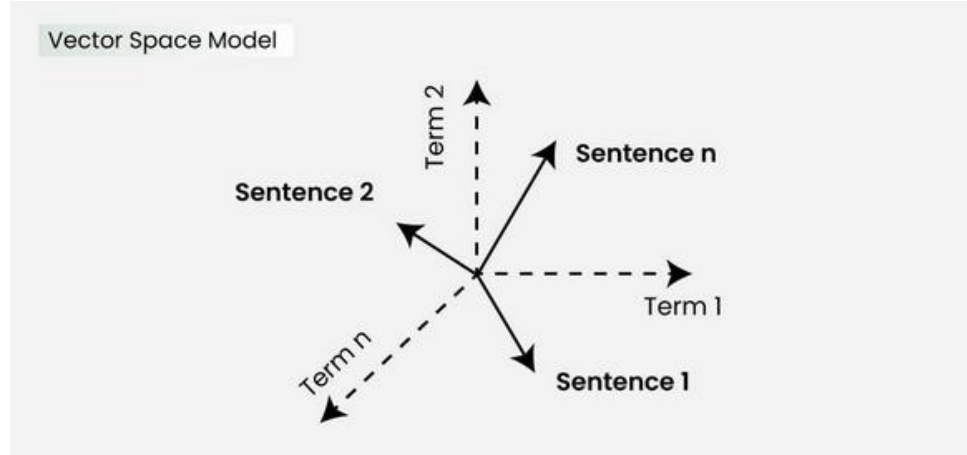
Boolean retrieval Algorithm



Keywords Searching Algorithms For Search Engine

4-Vector Space Model:

- **Algorithm Overview:** Represents documents and queries as vectors in a multi-dimensional space. The cosine similarity between vectors is used to measure the relevance of documents to a query.
- **Significance:** Allows for a more nuanced representation of document-query relationships, considering the similarity of terms and their weights.



Vector Space Model (VSM) Basic Terms

1. **Document-Term Matrix:** To create the vector representation of a collection of documents, you first construct a Document-Term Matrix (DTM) or Term-Document Matrix (TDM). Rows in this matrix represent documents, and columns represent terms (words or phrases). Each cell contains a numerical value representing a term's frequency or importance within a document.
2. **Term Frequency-Inverse Document Frequency (TF-IDF):** Once you have the DTM, you often apply a TF-IDF transformation to the raw term frequencies. TF-IDF stands for Term Frequency-Inverse Document Frequency and is a measure that reflects the importance of a term within a document relative to its importance across all documents in the corpus. It helps in highlighting important terms while downplaying common terms.
3. **Vectorization:** After TF-IDF or a similar transformation, each document is **represented as a vector** in the high-dimensional space. The length of these vectors is typically the same (equal to the number of unique terms in the corpus), but the values in each dimension vary depending on the term's importance within the document.
4. **Cosine Similarity:** To compare documents or perform text retrieval, you can use cosine similarity as a metric to measure the similarity between two document vectors. Cosine similarity calculates the cosine of the angle between two vectors and ranges from -1 (entirely dissimilar) to 1 (completely similar). A higher cosine similarity indicates a more significant similarity between documents.

Cosine Similarity in a Vector Space Model

Now that we understand how the Vector Space Model (VSM) represents text as vectors, it's time to explore one of the key concepts that make VSM so powerful in Natural Language Processing: **cosine similarity**.

What is Cosine Similarity?

Cosine similarity is a metric that measures the similarity between two vectors in a multi-dimensional space, such as the vectors representing documents in the VSM. In the context of VSM, it quantifies how alike two documents are based on their vector representations.

The key idea behind cosine similarity is to calculate the cosine of the angle between two vectors. If the vectors are very similar, their angle will be small, and the cosine value will be close to 1. Conversely, if the vectors are dissimilar, the angle will be large, and the cosine value will approach 0.

How is Cosine Similarity Calculated?

The formula for calculating cosine similarity between two vectors A and B is as follows, Where:

- $A \cdot B$ represents the dot product of vectors A and B.
- $\|A\|$ and $\|B\|$ represent the Euclidean norms (magnitudes) of vectors A and B, respectively.

The cosine similarity value ranges from -1 (completely dissimilar) to 1 (completely similar). A higher cosine similarity score indicates greater similarity between the two vectors.

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

Cosine Similarity in a Vector Space Model

In a VSM, cosine similarity is crucial for information retrieval and document ranking. Here's how it works in practice:

1. **Vector Representation:** We represent documents and queries as vectors using techniques like TF-IDF. Each document in the corpus and the query are converted into vectors in the same high-dimensional space.
2. **Cosine Similarity Calculation:** To determine the relevance of a document to a query, we calculate the cosine similarity between the query vector and the vectors representing each document in the corpus.
3. **Ranking:** Documents with higher cosine similarity scores to the query are considered more relevant and are ranked higher. Those with lower scores are ranked lower.

Vector Space Model example-1

Let's walk through a simple example of the Vector Space Model (VSM) using a small corpus of documents and a query. In this example, we'll represent documents and a query as vectors and calculate cosine similarity to retrieve relevant documents based on the query.

- **Step 1: Corpus الفهرس and Query**

Let's start with a small corpus of three documents and a query:

Document 1: "The quick brown fox jumps over the lazy dog."

Document 2: "A brown dog chased the fox."

Document 3: "The dog is lazy."

Query: "brown dog"

Vector Space Model example-2

- **Step 2: Create the Document-Term Matrix (DTM)**

We create a DTM where rows represent documents and columns represent terms. We'll use TF-IDF values for each term in the matrix:

Here, we've calculated TF-IDF values for each term in the documents and the query. You can use different formulas for TF-IDF, but this is common.

	a	brown	chase d	dog	fox	is	jumps	lazy	over	quick	the
Doc 1	0	0.29	0	0.29	0.29	0	0.29	0.29	0.29	0.29	0.58
Doc 2	0.41	0.29	0.41	0.29	0.29	0	0	0	0	0	0.41
Doc 3	0	0	0	0.41	0	0.41	0	0.41	0	0	0.41
Query	0	0.71	0	0.71	0	0	0	0	0	0	0

<https://spotintelligence.com/2023/09/07/vector-space-model/>

Vector Space Model example-3

- **Step 3: Vectorize the Query**

The query is also represented as a vector. In this case, it's a simple binary vector where 1 represents the presence of a term and 0 represents the absence:

	a	brown	chase d	dog	fox	is	jumps	lazy	over	quick	the
Query	0	1	0	1	0	0	0	0	0	0	0

Vector Space Model example-4

- **Step 4: Calculate Cosine Similarity**

Now, we calculate the cosine similarity between the query vector and each document vector. The formula for cosine similarity is:

$$\text{Cosine Similarity} = \frac{\text{Query Vector} \cdot \text{Document Vector}}{\|\text{Query Vector}\| \cdot \|\text{Document Vector}\|}$$

Using this formula, we calculate the cosine similarity between the query and each document:

- Cosine Similarity(Query, Doc 1) $\approx$ 0.58
- Cosine Similarity(Query, Doc 2) $\approx$ 0.29
- Cosine Similarity(Query, Doc 3) $\approx$ 0.41

Vector Space Model example-5

- **Step 5: Rank Documents by Similarity**

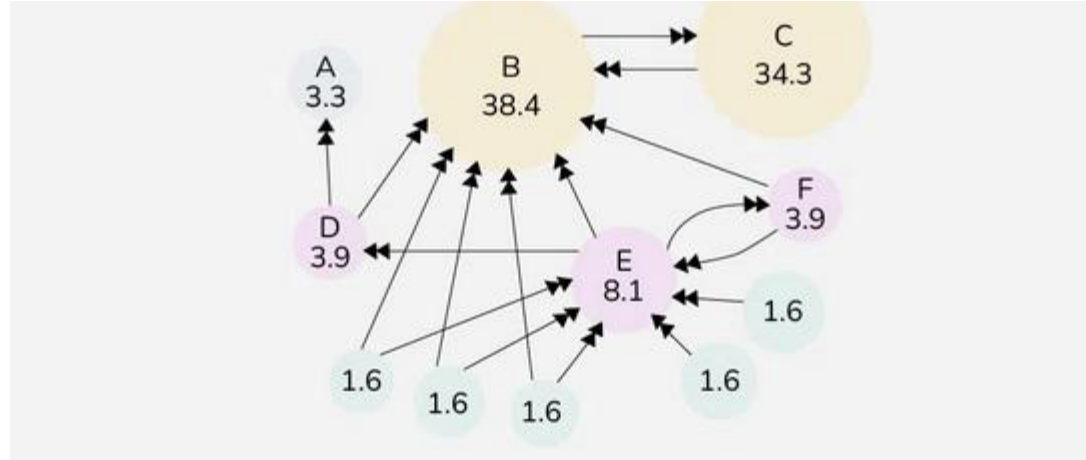
The documents are ranked by their cosine similarity values in descending order:

1. Document 1: Cosine Similarity ≈ 0.58
2. Document 3: Cosine Similarity ≈ 0.41
3. Document 2: Cosine Similarity ≈ 0.29

So, based on cosine similarity, Document 1 is the most relevant to the query “brown dog,” followed by Document 3 and then Document 2. This demonstrates how the Vector Space Model can be used for information retrieval and ranking documents based on their similarity to a query.

Pagerank Algorithm:

- **Algorithm Overview:**
Developed by Google, Pagerank Keyword Searching Algorithms evaluate the importance of web pages based on the number and quality of links pointing to them.
- **Significance:** Plays a crucial role in ranking search results, with more authoritative pages receiving higher rankings.



Page Rank Algorithm

- Google's first algorithms, which looks at links between pages to determine their relevance.
- PR is a number generated for each page available in Google Index
- Real Page rank is calculated based on number of pages in index, which can be 0.15 to Trillions

PR Formula

$$PR(A) = \frac{1 - d}{N} + d \left(\frac{PR(B)}{L(B)} + \frac{PR(C)}{L(C)} + \frac{PR(D)}{L(D)} + \dots \right).$$

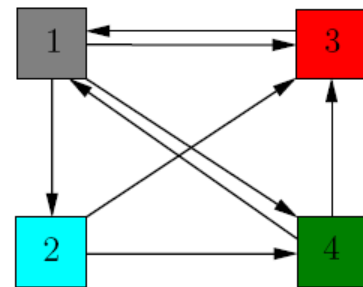
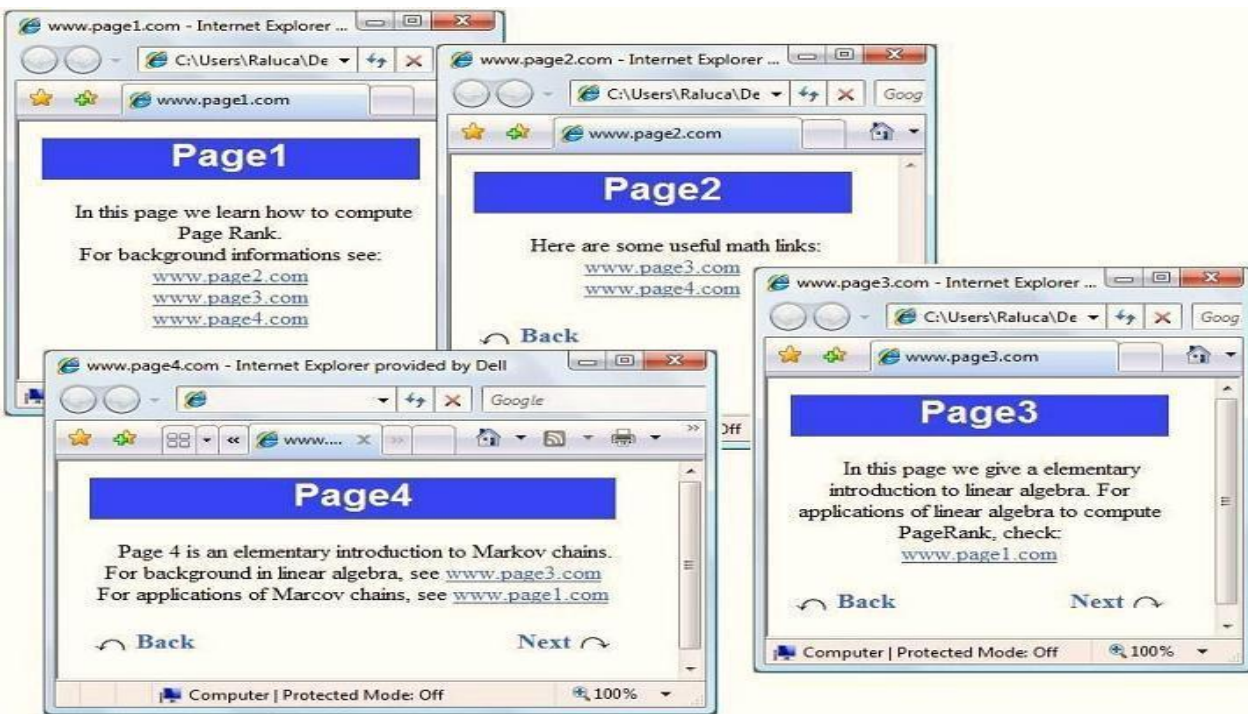
where

- **PR(A)** is the PageRank of page A,
- **PR(B..C..D)** is the PageRank of pages B/C/D which link to page A,
- **L(B..C..D)** is the number of **outbound** links on page B/C/D الروابط التي تشير لمواقع أخرى
- **d** is a **damping** factor which can be set between 0 and 1. رقم للتحكم في سرعة الحسابات
- **N** is the total number of **all pages** on the web

<http://pr.efactory.de/e-pagerank-algorithm.shtml>

Another Formula: $PR(A) = (1-d) + d \left(\frac{PR(Incoming Site)}{L(Incoming Site)} + \frac{PR(Outbound Links)}{L(Outbound Links)} \right)$

PageRank Example (Pages to graph)



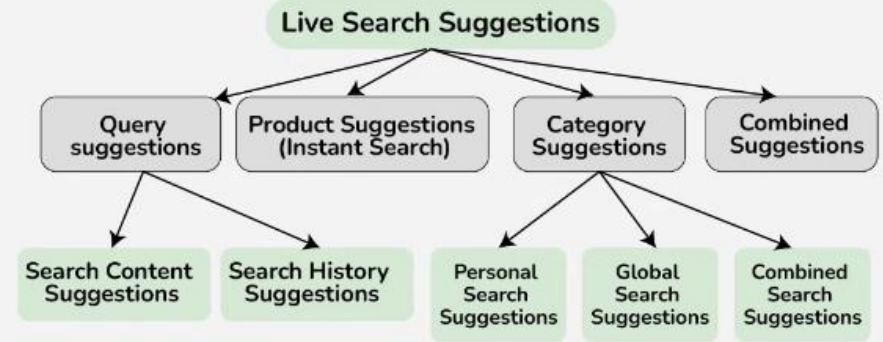
<http://www.math.cornell.edu/~mec/Winter2009/RalucaRemus/Lecture3/lecture3.html>

Autocomplete and Suggestions:

- **Algorithm Overview:** Predicts and suggests possible query completions based on partial user input. Utilizes techniques like Trie data structures and n-gram models.
- **Significance:** Enhances user experience by providing real-time suggestions, reducing the need for users to type complete queries.

<https://www.geeksforgeeks.org/keyword-searching-algorithms-for-search-engines/>

Suggestion and Autocorrection

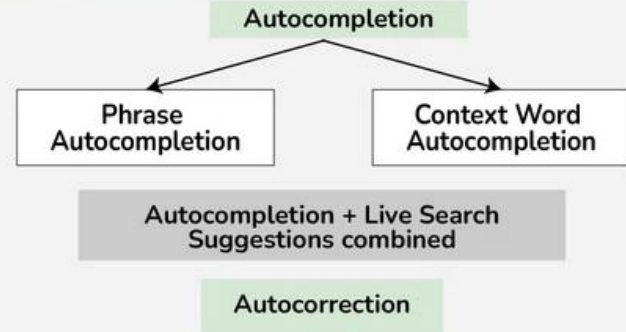


Keyword Searching Algorithms



Suggestions Algorithm

Suggestion and Autocorrection



Spell Checking

- Important part of query processing
 - 10-15% of all web queries have spelling errors
- Errors include typical word processing errors but also many other types, e.g.

poiner sisters
brimingham news
catamarn sailing
hair extenssions
marshmellow world
miniture golf courses
psychics
home doceration

realstateisting.bc.com
akia 1080i manunal
ultimatwarcade
mainsourcebank
dellottitouche

Spell Checking

- Basic approach: suggest corrections for words not found in *spelling dictionary*
- Suggestions found by comparing word to words in dictionary using similarity measure
- Most common similarity measure is *edit distance*
 - number of operations required to transform one word into the other

Edit Distance

- *Damerau-Levenshtein* distance

- counts the minimum number of insertions, deletions, substitutions, or transpositions of single characters required
- e.g., Damerau-Levenshtein distance 1

- distance 2

extenssions → extensions (insertion error)

poiner → pointer (deletion error)

marshmellow → marshmallow (substitution error)

brimingham → birmingham (transposition error)

doceration → deceration

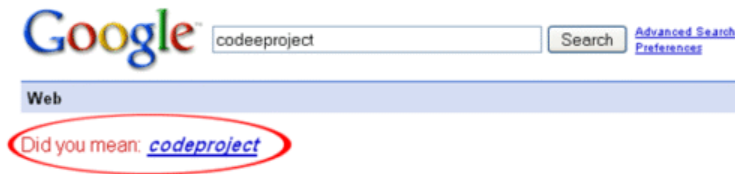
deceration → decoration

Edit Distance

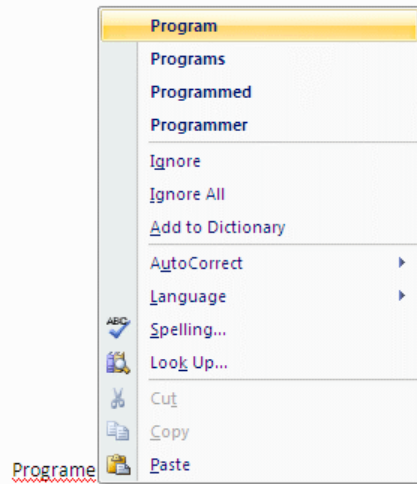
- Number of techniques used to speed up calculation of edit distances
 - restrict to words starting with same character
 - restrict to words of same or similar length
 - restrict to words that sound the same
- Last option uses a *phonetic code* to group words
 - e.g. Soundex

Soundex introduction

- Soundex is a phonetic algorithm for indexing names by sound. Many applications use algorithms like this to add fantastic features like the Google Spelling Corrections and MS Word Autocorrect.



<https://www.codeproject.com/Articles/26880/Arabic-Soundex>



Soundex Code

1. Keep the first letter (in upper case).
2. Replace these letters with hyphens: a,e,i,o,u,y,h,w.
3. Replace the other letters by numbers as follows:
 - 1: b,f,p,v
 - 2: c,g,j,k,q,s,x,z
 - 3: d,t
 - 4: l
 - 5: m,n
 - 6: r
4. Delete adjacent repeats of a number.
5. Delete the hyphens.
6. Keep the first three numbers or pad out with zeros.

extenssions → E235; extensions → E235

marshmellow → M625; marshmallow → M625

brimingham → B655; birmingham → B655

poiner → P560; pointer → P536

Arabic Soundex

Word 1 Start Word 2

Select a word

id	Name	Value
1	أحمد	x530
2	أحمد	x530
3	أحمد	x530
4	أحمد	x535
5	أحمد	x550
6	أحمد	x550
7	أحمد	x535
8	أحمد	x550
9	أحمد	x550
10	أحمد	x354
11	أحمد	x344
12	أحمد	x354
13	أحمد	x543
14	أحمد	x540
15	أحمد	x560
16	أحمد	x560
17	أحمد	x540
18	أحمد	x540
19	أحمد	x610
20	أحمد	x610
21	أحمد	x530

Similar Words

id	Name	Value
1	أحمد	x530
2	أحمد	x530
3	أحمد	x530
21	أحمد	x530

☒ Arabic ☐ English

Create Sound List

Insert

<https://www.codeproject.com/Articles/26880/Arabic-Soundex>

Arabic Soundex-2

- The Soundex algorithm stands on grouping similar sounding letters depending on special sounding features, as follows:

Soundex Code	المحارف	المجموعة
1	b, f, p, v	شَقَهِيّ
2	c, g, j, k, q, s, x, z	حروف حَلْقِيَّة و حُرُوف صَفِير
3	d, t	حرف نَطْعِي (ملفوظ بوضع اللسان على مؤخر الاسنان الامامية العليا)
4	l	حرف صامت ملفوظ بلطف طَوِيل
5	m, n	حرف يلفظ من الانف
6	r	حرف صامت ملفوظ بلطف قَصِير

Arabic Soundex-3

1. Hold the first letter.
2. Replace the characters (أ, إ, ؤ, ح, ع, غ, ش, ي) with the value 0.
3. Replace the characters (ب, ف) with the value 1.
4. Replace the **characters** (ك, ق, ظ, ص, س, ز, ج, خ) with the value 2.
5. Replace the characters (ط, ض, ذ, ث, ت) with the value 3.
6. Replace the character (ل) with the value 4.
7. Replace the characters (ن, م) with the value 5.
8. Replace the character (ر) with the value 6.
9. Remove the (أ, إ, ؤ) characters from the **beginning** of the word if found, since they added more confusion.
10. Ignore the first character handling: in English language, it is important to handle the first character, but not always in Arabic
11. Update the character sound categories by removing or adding some characters.

Polite Crawlers

In the early days of web crawlers there was no protocol about how often to request pages, or which pages to include, so some crawlers requested entire sites at once, putting stress on the servers.

- **Robots Exclusion Standard** can help webmasters block certain pages from being crawled and indexed

لا يفحص كل صفحات الموقع ، ولكن ما يتم السماح به بشكل صريح من جانب الموقع

Robots Exclusion Standard

The Robots Exclusion Standard is implemented with plain text files named **robots.txt** stored at the root of the domain.

Robots.txt has two syntactic **elements** for each **rule**:

1. **user-agent** (crawler name) we want to make a **rule** for (the special character * means all agents)
2. **One Disallow** directive per line to identify patterns. تعريف
بالمكان الممنوع على هذا المتصفح

Robots Exclusion Standard

Robots.txt

The Robots Exclusion Standard is not a layer of security.

Could be important for web programmers of big sites

Some malicious bots will not obey the directives and purposefully seek out materials specifically disallowed in **robots.txt**.

```
User-agent: googlebot  
Disallow:
```

Allow all , could just have no file

```
User-agent: funbot  
Disallow: /secret/
```

Please don't search in the secret directory

```
User-agent: *  
Disallow: /
```

Please don't search here at all.

LISTING 20.2 Robots.txt to allow googlebot full access, allow funbot partial access, and block all other bots

List of common agents names (robots.txt)

<https://www.woorank.com/en/edu/seo-guides/robots-txt>

User-Agent

baiduspider
baiduspider-image
baiduspider-mobile
baiduspider-news
baiduspider-video
bingbot
msnbot
msnbot-media
adidxbot
Googlebot
Googlebot-Image
Googlebot-Mobile
Googlebot-News
Googlebot-Video
Mediapartners-Google
AdsBot-Google
slurp
yandex

Search Engine

Baidu
Baidu
Baidu
Baidu
Baidu
Bing
Bing
Bing
Bing
Google
Google
Google
Google
Google
Google
Google
Yahoo!
Yandex

Field

General
Images
Mobile
News
Video
General
General
Images & Video
Ads
General
Images
Mobile
News
Video
AdSense
AdWords
General
General

Concurrency (multithreading)

عمل أكثر من وظيفة في نفس الوقت

- A crawler incurs several delays:
 - Resolving the host name in the URL to an IP address using DNS
 - Connecting a socket to the server and sending the request
 - Receiving the requested page in response
- Solution: Overlap the above delays by **fetching many pages concurrently**

Parallel Search Algorithm

Binary search is an example of divide and conquer algorithm.

Pseudocode

Divide and Conquer

In divide and conquer approach, the problem is divided into several small sub-problems. Then the sub-problems are solved recursively and combined to get the solution of the original problem.

The divide and conquer approach involves the following steps at each level –

- **Divide** – The original problem is divided into sub-problems.
- **Conquer** – The sub-problems are solved recursively.
- **Combine** – The solutions of the sub-problems are combined to get the solution of the original problem.

```
Binarysearch(a, b, low, high)

if low < high then
    return NOT FOUND
else
    mid ← (low+high) / 2
    if b = key(mid) then
        return key(mid)
    else if b < key(mid) then
        return BinarySearch(a, b, low, mid-1)
    else

        return BinarySearch(a, b, mid+1, high)
```


References

Search engine introduction

<https://www.geeksforgeeks.org/keyword-searching-algorithms-for-search-engines/>

<https://www.studysmarter.co.uk/explanations/computer-science/computer-network/search-engine-indexing/>

Parallel search

https://www.tutorialspoint.com/parallel_algorithm/parallel_search_algorithm.htm

<https://spotintelligence.com/2023/09/07/vector-space-model/>